

Apollo TEMP_PRO-1 Sensor — Operations Guide

A field reference for deploying, configuring, and troubleshooting the TEMP_PRO-1 MQTT (Ethernet and WiFi variants).

Table of Contents

1. What It Is
 2. Hardware Specs
 3. Power Modes (Battery vs Wired)
 4. Deployment Checklist
 5. Boot Button Cheat Sheet
 6. Web UI Reference
 7. MQTT Topic Reference
 8. Safety Features
 9. Common Operator Workflows
 10. Troubleshooting
-

What It Is

A self-contained air-quality sensor that wakes every 30 minutes (configurable), samples PM/temperature/humidity, publishes to MQTT, and goes back to sleep. On battery alone it lasts roughly **5 months** between charges. Plugged into PoE/USB it runs continuously.

Built around an ESP32-S3 with deep-sleep, an SEN55 PM/VOC/NOx sensor, an SHT3x temperature/humidity sensor, an SHT3x probe input, and a MAX17048 battery fuel gauge.

Hardware Specs

Component	Spec
MCU	ESP32-S3, 160 MHz (down-clocked from 240 for power)
PM/VOC/NOx	Sensirion SEN55 (mass concentration only — VOC/NOx disabled for short wakes)
Temp/Humidity	SHT3x (probe-mountable for accuracy)
Battery gauge	MAX17048 (always-on)
Battery	8000 mAh LiPo
Network	W5500 Ethernet (PoE-capable) or WiFi (separate firmware)
Connectivity	MQTT only — no Home Assistant API, no auto-discovery
Power options	USB-C, PoE (Ethernet variant), battery
Sleep current	~6.9 mA (with W5500) / ~4.2 mA (without) / ~14 µA chip-floor
Active current	~150–250 mA depending on workload

Power Modes (Battery vs Wired)

The single most important setting. Determined by `Power Source` (web UI) or `apollo/temp-pro-1/command/power_source` (MQTT).

`battery` mode (default)

- Wakes on a timer (`Sleep Duration` , default 30 min)
- Connects to MQTT, publishes one batch of readings, sleeps
- Forced sleep at `Max Wake Time` (default 45 s) regardless of MQTT state

- Critical-battery handling: < 5% battery extends sleep to 60 min
- Drain detection running in background

wired mode

- Stays awake continuously
- Re-publishes every `Publish Interval` seconds (default 10 s)
- Drain detection running in background — flips to battery if drain detected
- Boot-loop detection running — flips to battery if 3+ cold boots in a row

Auto-flip safety

A `wired` device flips to `battery` automatically if: - Drain rate ≥ 1.5 %/h **AND** drop $\geq 3\%$ from baseline (rate trigger) - Drop $\geq 5\%$ from baseline regardless of rate (cumulative trigger) - Battery hits critical threshold (default 5%) - 3+ consecutive cold boots detected (boot loop)

Operator returns it to wired mode (if intended) by web UI, MQTT command, or the boot button.

Deployment Checklist

Per unit, at install time

1. **Flash firmware** (or use pre-flashed unit)
2. **Power on** (battery or USB)
3. **Set Power Source - Battery deploys:** Leave default (`battery`). Skip this step. - **Wired deploys:** 3× boot button (or web UI / MQTT) → `wired`
4. **Set Column ID** (web UI: `Column ID` dropdown or `Custom Column ID` text field) — used for per-device topic prefix
5. **Confirm via logs or first MQTT publish:** - `Power Src:` matches reality - `Drain Alarm:` OK - `Health:` OK - Sensors returning real values

Per fleet, before deploying

1. Set **MQTT broker / port / username / password** — either via the `!secret` defaults at compile time, or per-device via the web UI text fields (override compile-time defaults)
 2. Set **Web Auth Username / Password** if you don't want defaults
 3. Set static IP if needed (web UI: `Use Static IP` switch + IP fields)
 4. Set deployment metadata (`Deployment Notes`)
-

Boot Button Cheat Sheet

Single button on the device, multi-function based on press pattern:

Press	Action	LED Feedback
1x short	Status check	Brief color flash:  connected to MQTT,  WiFi only,  nothing
3x clicks	Toggle Power Source (battery ↔ wired)	Solid 3 s:  wired,  battery
Hold 10 s	Factory reset (clears all NVS)	—

Note: pressing the button while the device is in deep sleep wakes it up (any press). Press patterns are recognized only while awake.

Web UI Reference

Access via `http://apollo-tmp-pro-1-mqtte-<mac>.local/` (or by IP). Default credentials: `admin` / `tmna` (override via `Web Auth Username` / `Web Auth Password`).

Configuration

Setting	Default	Notes
Power Source	battery	battery / wired
Column ID	Not Set	Dropdown of pre-defined IDs (A1... M13)
Custom Column ID	empty	Text override; takes precedence if non-empty
Sleep Duration	30 min	Battery wake interval
Max Wake Time	45 s	Battery hard cap before forced sleep
Warm-up Delay	20 s	SEN55 fan stabilization before sample
Publish Interval	10 s	Wired-mode re-publish rate
Drain Alarm Threshold	1.5 %/h	Drain rate trigger
Drain Alarm Min Drop	3 %	Drain rate-trigger noise floor
Wired Drop Threshold	5 %	Cumulative drop trigger (wired-only)
Critical Battery Threshold	5 %	Trips critical-battery handler
Critical Battery Sleep Duration	60 min	Sleep extension when critical
Battery LED on Wake	OFF	Show battery color LED on each wake
Use Static IP	OFF	Apply static IP on next reboot
Static IP / Gateway / Subnet / DNS1 / DNS2	empty	Used if static IP enabled
MQTT Broker / Port / Username / Password	empty	Override <code>!secret</code> defaults
Web Auth Username / Password	empty	Override compiled defaults
Deployment Notes	empty	Free text for tech notes
Verbose Logging	ON	DEBUG-level logs (off = WARN only)

Diagnostic — Read-only

Sensor	Source
Battery level / voltage	MAX17048
Temperature / Relative Humidity	SHT3x probe
ESP Temperature	Internal die temp
PM 0.3-1 / 1-2.5 / 2.5-4 / 4-10 / Total	SEN55
Lifetime Min/Max Battery + Voltage	NVS-tracked
Drain Trigger Count	NVS-tracked counter
SEN55 / SHT3x / MAX17048 Fail Count	Consecutive read failures
Drain Alarm Triggered (this wake)	Live binary flag
Critical Battery (this wake)	Live binary flag
Wake Reason	timer / button / cold_boot / other
Health Status	OK / DEGRADED / CRITICAL
Last Drain Trigger	"Never" or "Boot #N (M wakes ago)"
Uptime	Since this wake
Device ID	MAC address
IP Address	Current network IP

Buttons

Button	Effect
ESP Reboot	Soft restart
Publish Now	Force a <code>publish_mqtt_data</code> run
Force Sleep	End wake immediately + flip to battery
Clean SEN55 Fan	Trigger SEN55 fan cleaning cycle
Test Buzzer / Stop Buzzer	Sound test
Restart SEN55	Recover wedged sensor without rebooting device
Test LED	Cycle R/G/B to verify hardware

MQTT Topic Reference

All topics use the prefix `apollo/temp-pro-1/<device-id>/` where `<device-id>` is the configured Column ID (or MAC suffix if unset).

Sensor Data (published per wake)

Topic	Type	Notes
<code>sensor/pm_<range></code>	float ($\mu\text{g}/\text{m}^3$)	PM mass for sub-ranges 0.3-1, 1-2.5, 2.5-4, 4-10
<code>sensor/total_pm</code>	float ($\mu\text{g}/\text{m}^3$)	Cumulative PM $\leq 10 \mu\text{m}$
<code>sensor/temperature</code>	float ($^{\circ}\text{F}$)	SHT3x reading (-1 sentinel if NaN)
<code>sensor/humidity</code>	float (%)	SHT3x reading (-1 sentinel if NaN)
<code>sensor/battery</code>	int (%)	MAX17048 SoC
<code>sensor/battery_voltage</code>	float (V)	MAX17048 voltage

Meta — Static / Slow-Changing (retained)

Topic	Type	Notes
meta/online	"true" / "false"	Per-device LWT — auto-published "false" by broker on disconnect
meta/power_source	"battery" / "wired"	Current power mode
meta/sleep_mode	"ON" / "OFF"	Legacy compat — derived from power_source
meta/firmware_version	string	Project version
meta/boot_count	int	Lifetime wake counter
meta/wake_reason	string	timer / button / cold_boot / other_N
meta/free_heap	int (KB)	RAM free at publish time
meta/uptime_s	int	Seconds since this wake
meta/wake_time_s	float	Same as uptime, float
meta/sen55_serial	string	SEN55 hardware ID
meta/sen55_fw	string	SEN55 firmware version
meta/ip_address	string	Current network IP
meta/mac_address	string	Device MAC
meta/column_id	string	Configured column ID
meta/notes	string	Deployment notes field
meta/device_name	string	ESPHome device name

Meta — Diagnostic Counters (retained)

Topic	Notes
<code>meta/drain_trigger_count</code>	Lifetime count of drain alarm trips
<code>meta/drain_last_trigger_boot</code>	Boot number when alarm last fired (0 = never)
<code>meta/sen55_fail_count</code> / <code>meta/sht_fail_count</code> / <code>meta/max17048_fail_count</code>	Consecutive NaN reads (cleared on success)
<code>meta/lifetime_min_battery_pct</code> / <code>_max_battery_pct</code>	Lowest/highest battery % seen
<code>meta/lifetime_min_battery_v</code> / <code>_max_battery_v</code>	Lowest/highest voltage seen

Meta — Configured Values (retained)

Topic	Notes
<code>meta/config_sleep_duration_min</code>	Configured sleep duration
<code>meta/config_max_wake_time_s</code>	Configured wake-time cap
<code>meta/config_drain_threshold_pct_per_h</code>	Configured drain rate threshold
<code>meta/config_drain_min_drop_pct</code>	Configured drain min drop
<code>meta/config_wired_drop_threshold_pct</code>	Configured wired-cumulative trigger
<code>meta/config_critical_battery_threshold_pct</code>	Configured critical battery threshold
<code>meta/config_critical_battery_sleep_min</code>	Configured critical battery sleep
<code>meta/config_warmup_delay_s</code>	Configured SEN55 warmup
<code>meta/config_publish_interval_s</code>	Configured wired publish interval

Use these for fleet config audit:

```
mosquitto_sub -t '+/meta/config_drain_threshold_pct_per_h' -v
```

Status — Health Flags (retained)

Topic	Values	Notes
<code>status/health</code>	OK / DEGRADED / CRITICAL	Single-glance rollup
<code>status/drain_protection</code>	OK / TRIGGERED	TRIGGERED only during the wake the alarm fired
<code>status/sen55_errors</code>	hex string	SEN55 device status register (0 = healthy)

Units — Reference

Topic	Value
<code>meta/unit_pm</code>	"ug/m3"
<code>meta/unit_battery</code>	"%"
<code>meta/unit_temperature</code>	"F"
<code>meta/unit_humidity</code>	"%"

Commands (accepted topics, all under `apollo/temp-pro-1/`)

Three flavors of every command for fleet targeting: - `command/<cmd>` — every device - `battery/command/<cmd>` — battery devices only - `wired/command/<cmd>` — wired devices only

Command	Payload	Notes
<code>power_source</code>	<code>battery</code> / <code>wired</code> (or <code>ON</code> / <code>OFF</code>)	Set power mode
<code>prevent_sleep</code>	<code>ON</code> / <code>OFF</code>	Legacy — <code>ON</code> → <code>wired</code> , <code>OFF</code> → <code>battery</code>
<code>sleep_duration</code>	minutes (1–1440)	Set wake interval
<code>restart</code>	(any)	Reboot device
<code>light/on</code> / <code>light/off</code>	(any)	RGB LED
<code>light/red</code> / <code>light/green</code> / <code>light/blue</code>	(any)	RGB LED solid color
<code>light/rgb</code>	"r,g,b" 0-255	RGB LED custom
<code>light/effect</code>	"Slow Pulse" / "Fast Pulse" / "none"	RGB LED animation
<code>buzzer</code>	RTTTL string OR "beep" / "siren" / "success" / "error"	Play sound
<code>buzzer/stop</code>	(any)	Stop sound
<code>column_id</code>	string	Set Column ID
<code>custom_column_id</code>	string	Set custom column ID text
<code>deployment_notes</code>	string	Set notes field
<code>warmup_delay</code>	seconds	Set SEN55 warmup
<code>max_wake_time</code>	seconds	Set wake cap
<code>battery_led_on_wake</code>	<code>ON</code> / <code>OFF</code>	Toggle wake-time LED
<code>publish_interval</code>	seconds	Set wired publish interval
<code>mqtt_broker</code> / <code>mqtt_port</code> / <code>mqtt_username</code> / <code>mqtt_password</code>	string / int	Override broker config (takes effect after reboot)

Command	Payload	Notes
<code>web_auth_username /</code> <code>web_auth_password</code>	string	Override web auth (after reboot)

Safety Features

The device has overlapping protections for accidental drain. From most restrictive to least:

1. Wake-time cap (battery devices)

If `power_source = battery`, the device **always** sleeps within `Max Wake Time` (default 45 s). No MQTT command, no stuck retained message, no firmware bug can keep it awake longer.

2. Drain detector (all devices)

Every wake (battery) or every 5 min (wired), checks battery drop and rate:

- **Rate trigger:** drop $\geq 3\%$ AND rate $\geq 1.5\%/h$ → flip to battery
- **Wired-cumulative trigger:** drop $\geq 5\%$ on a wired device → flip to battery

When triggered: - `power_source` flipped to `battery` - `drain_trigger_count` incremented - `status/drain_protection` published as `TRIGGERED` - Device sleeps on next safety-net cycle

3. Critical battery (all devices)

If battery drops below `Critical Battery Threshold` (default 5%): - Flip `power_source` to `battery` if not already - Extend next sleep to `Critical Battery Sleep Duration` (default 60 min) - Publish `status/critical_battery = TRIGGERED` - Still publishes data each wake (operator sees the alert)

4. Boot-loop detection

3+ consecutive `cold_boot` wakes (no normal timer wake in between): - Detected as a crash loop - Flip to battery - Force 30-min cooldown sleep - Counter clears on the first healthy timer/button wake

5. LWT (Last Will & Testament)

On every connect: device publishes `meta/online = true` . If the connection drops ungracefully (network outage, power loss), the broker auto-publishes `meta/online = false` . Subscribers can tell "device offline" from "device sleeping but alive."

6. Retained-command scrub

On every successful MQTT connect, the device clears any retained command messages at the broker. Defends against the original "stale retained prevent_sleep=ON drained the fleet overnight" failure mode.

Common Operator Workflows

Deploy a new battery unit (most common)

1. Flash firmware (or use pre-flashed unit)
2. Power on
3. Open web UI, set `Column ID` , save
4. Done. Device wakes every 30 min, publishes, sleeps.

Deploy a wired (PoE/USB) unit

Same as above, plus: - 3x boot button (or web UI) to set `Power Source` = wired - Confirm log shows `Power Src: wired`

Wake up battery devices for live debugging

Subscribe to `+/meta/online` to see who's connected. Then publish:

```
mosquitto_pub -t apollo/temp-pro-1/battery/command/power_source -m wired
```

The retained message stays at the broker. Each battery device, on its next 30-min wake, picks it up and flips to `wired` mode (stays awake).

When done, clear the command:

```
mosquitto_pub -t apollo/temp-pro-1/battery/command/power_source -m "" -r
```

Then explicitly send each device back to battery:

```
mosquitto_pub -t apollo/temp-pro-1/battery/command/power_source -m batte
```

Find unhealthy units in a fleet

```
# All currently CRITICAL
mosquitto_sub -t '+/status/health' -v -W 5 | grep CRITICAL

# All that have ever tripped drain alarm
mosquitto_sub -t '+/meta/drain_trigger_count' -v -W 5 | awk '$2 != "0"'

# All currently offline (per LWT)
mosquitto_sub -t '+/meta/online' -v -W 5 | grep false
```

Audit fleet config

Confirm all devices have the same drain threshold:

```
mosquitto_sub -t '+/meta/config_drain_threshold_pct_per_h' -v -W 5
```

Identify drained-then-recovered units

These units have `drain_trigger_count` > 0 but currently `health = OK` :

```
# Subset 1: had drain history
mosquitto_sub -t '+/meta/drain_trigger_count' -v -W 5 | awk '$2 != "0"'

# Cross-reference with currently OK
mosquitto_sub -t '+/status/health' -v -W 5 | grep OK
```

Fully reset a single unit

10-second hold on the boot button → factory reset. Wipes all NVS including Column ID, MQTT settings, lifetime stats. Use only when fully redeploying.

Troubleshooting

Device not publishing to MQTT

Check, in order: 1. **Network**: device's web UI shows IP? Ethernet link LEDs? 2. **DNS**: log shows `Couldn't resolve IP address for <broker>` ? → DNS misconfigured. Use IP directly via `MQTT Broker` web UI override. 3. **Auth**: log shows authentication errors? → Check broker username/password. 4. **Firewall**: device on different VLAN than broker? → Open port 1883. 5. **Sleeping**: battery device just sleeping. Wait for next wake (≤30 min).

Battery dies faster than expected

Check `Drain Trigger Count` per device. Non-zero means the drain detector caught something. Subscribe to `meta/wake_reason` — lots of `cold_boot` indicates crashes. Check `Lifetime Min Battery` for actual low watermark.

"Health: DEGRADED" right after a reboot

Expected. The boot button reset increments `rapid_boot_count`, which flags DEGRADED briefly. Clears automatically on the next healthy sensor read (~30 min on battery, ~5 min on wired).

"Health: CRITICAL" right after firmware update

Most likely the boot-loop detector hit 3 cold boots (firmware updates + reboots count). Wait 30 min — one normal timer wake clears it. Or 10s boot-button factory reset for an immediate clear.

Drain alarm fires too often / too rarely

Tune in web UI: - More sensitive: lower `Drain Alarm Threshold` (default 1.5 %/h) or `Drain Alarm Min Drop` (default 3%) - Less sensitive: raise both - For wired-specific: `Wired Drop Threshold` (default 5%)

Lifetime min battery shows skewed value (e.g., 0%) from a prior incident

Cosmetic — doesn't affect operation. Will normalize over time. Or factory reset for a clean slate.

"Wired Drop Threshold" trips on a healthy unit

The MAX17048 has 1-2% noise per reading, but baseline only updates on $\geq 5\%$ rises. If a wired unit charged from low and drifted while staying near full, the baseline could be slightly below current. Acceptable false positive; will normalize on next charge cycle.

Boot button doesn't wake device

If the boot button doesn't wake from deep sleep: - Check the unit isn't dead (LiPo at 0%)
- Check the boot button isn't physically stuck - Try external reset (RST pin) — that's a hard cold-boot

MQTT broker IP changed

Web UI: update `MQTT Broker` field. Reboot via `ESP Reboot` button. Or via MQTT (works only if currently connected to old broker):

```
mosquitto_pub -t apollo/temp-pro-1/<id>/command/mqtt_broker -m new-broke
```

```
mosquitto_pub -t apollo/temp-pro-1/<id>/command/restart -m 1
```

Device stuck in wired mode but is supposed to be battery

The drain detector will catch it eventually (~30 min – 2 hr depending on load). Or: - Web UI: change **Power Source** to **battery** - 3x boot button - MQTT: `mosquitto_pub -t apollo/temp-pro-1/<id>/command/power_source -m battery`

Device is in battery mode but should be wired

The drain detector flipped it. Check **Drain Trigger Count** — non-zero confirms. Investigate why drain happened (lost PoE? bad USB cable?) before flipping back, or it'll just trigger again.

Reference: Compile + Flash

```
# Compile only (no flash)
esphome compile TEMP_PRO-1_MQTT_ETH.yaml

# Flash via USB
esphome upload TEMP_PRO-1_MQTT_ETH.yaml

# Flash over network (OTA)
esphome upload TEMP_PRO-1_MQTT_ETH.yaml --device <device-ip>

# Stream live logs (with timestamps from your laptop)
esphome logs TEMP_PRO-1_MQTT_ETH.yaml

# Erase all flash (full clean slate including NVS)
esptool.py --port /dev/cu.usbmodemXXXX erase_flash
```

Appendix: Topic Examples

Replace `<id>` with the device's Column ID or MAC suffix.

```
# Subscribe to all data from one device
mosquitto_sub -t 'apollo/temp-pro-1/<id>/#' -v

# Subscribe to all data from all devices
mosquitto_sub -t 'apollo/temp-pro-1/+/#' -v

# Watch all devices' battery percentages
mosquitto_sub -t 'apollo/temp-pro-1/+/sensor/battery' -v

# Watch all devices' health rollups
mosquitto_sub -t 'apollo/temp-pro-1/+/status/health' -v

# Send wake-up to all battery devices
mosquitto_pub -t 'apollo/temp-pro-1/battery/command/power_source' -m wake

# Restart everyone
mosquitto_pub -t 'apollo/temp-pro-1/command/restart' -m 1

# Set sleep duration to 1 hour for all battery devices
mosquitto_pub -t 'apollo/temp-pro-1/battery/command/sleep_duration' -m 60
```